

Zeal College of Engineering and Research

Subject: LP-V

Name: Jenil Girish Rathod

Class: BE

Div: B

Batch: B1

Roll No: B22009

Group A: Practical No:3

Program Statement: Implement HPC application for AI/ML domain.

Implemented Code:

```
#include <iostream>
#include <fstream>
#include <vector>
#include <string>
#include <sstream>
#include <chrono>
#include <cmath>
#include <algorithm>
#include <numeric>
#include <omp.h>
using namespace std;
using Vec = vector<double>;
using Mat = vector<Vec>;

Mat read_csv(const string &path) {
    ifstream in(path);
    if (!in.is_open()) {
        cerr << "Failed to open " << path << endl;
        exit(1);
    }
    string line;
    Mat data;
    while (getline(in, line)) {
        if (line.empty()) continue;
        stringstream ss(line);
        string token;
        Vec row;
        bool ok = true;
        while (getline(ss, token, ',')) {
            try {
                double v = stod(token);
                row.push_back(v);
            } catch (...) {
                ok = false;
                break;
            }
        }
        data.push_back(row);
    }
    return data;
}
```

```

    }
    }
    if (ok && !row.empty()) data.push_back(row);
}
return data;
}

```

```

inline double sq_dist(const Vec &a, const Vec &b) {
    double s = 0.0;
    for (size_t i = 0; i < a.size(); ++i) {
        double d = a[i] - b[i];
        s += d * d;
    }
    return s;
}

```

```

struct KMeansResult {
    Mat centroids;
    vector<int> labels;
    double inertia;
    double elapsed;
};

```

```

KMeansResult kmeans_sequential(const Mat &data, Mat centroids, int max_iter = 100,
double tol = 1e-4) {
    using clock = chrono::high_resolution_clock;
    auto t0 = clock::now();
    int N = (int)data.size();
    int D = (int)data[0].size();
    int k = (int)centroids.size();
    vector<int> labels(N, -1);
    for (int it = 0; it < max_iter; ++it) {
        Mat sums(k, Vec(D, 0.0));
        vector<int> counts(k, 0);
        for (int i = 0; i < N; ++i) {
            int best = -1;
            double bestd = numeric_limits<double>::infinity();
            for (int j = 0; j < k; ++j) {
                double d = sq_dist(data[i], centroids[j]);
                if (d < bestd) { bestd = d; best = j; }
            }
            labels[i] = best;
            counts[best]++;
            for (int d = 0; d < D; ++d) sums[best][d] += data[i][d];
        }
        Mat newc = centroids;
        for (int j = 0; j < k; ++j) {
            if (counts[j] > 0) {
                for (int d = 0; d < D; ++d) newc[j][d] = sums[j][d] / counts[j];
            }
        }
    }
}

```

```

    }
    double shift = 0.0;
    for (int j = 0; j < k; ++j) shift = max(shift, sqrt(sq_dist(newc[j], centroids[j])));
    centroids.swap(newc);
    if (shift <= tol) break;
}
// compute inertia
double inertia = 0.0;
for (int i = 0; i < N; ++i) inertia += sq_dist(data[i], centroids[labels[i]]);
auto t1 = clock::now();
double elapsed = chrono::duration<double>(t1 - t0).count();
return {centroids, labels, inertia, elapsed};
}

```

KMeansResult kmeans_openmp(const Mat &data, Mat centroids, int max_iter = 100, double tol = 1e-4) {

```

    using clock = chrono::high_resolution_clock;
    auto t0 = clock::now();
    int N = (int)data.size();
    int D = (int)data[0].size();
    int k = (int)centroids.size();
    vector<int> labels(N, -1);
    int num_threads = omp_get_max_threads();
    for (int it = 0; it < max_iter; ++it) {
        // per-thread partial sums
        vector<vector<double>>> partial_sums(num_threads, vector<double>(k * D, 0.0));
        vector<vector<int>>> partial_counts(num_threads, vector<int>(k, 0));
        vector<double> partial_inertia(num_threads, 0.0);

```

#pragma omp parallel for schedule(static)

```

    for (int i = 0; i < N; ++i) {
        int tid = omp_get_thread_num();
        int best = -1;
        double bestd = numeric_limits<double>::infinity();
        for (int j = 0; j < k; ++j) {
            double d = 0.0;
            for (int dd = 0; dd < D; ++dd) {
                double diff = data[i][dd] - centroids[j][dd];
                d += diff * diff;
            }
            if (d < bestd) { bestd = d; best = j; }
        }
        labels[i] = best;
        partial_counts[tid][best] += 1;
        partial_inertia[tid] += bestd;
        for (int dd = 0; dd < D; ++dd) partial_sums[tid][best * D + dd] += data[i][dd];
    }
    Mat sums(k, Vec(D, 0.0)); // combine
    vector<int> counts(k, 0);
    double total_inertia = 0.0;

```

```

    for (int t = 0; t < num_threads; ++t) {
        total_inertia += partial_inertia[t];
        for (int j = 0; j < k; ++j) {
            counts[j] += partial_counts[t][j];
            for (int dd = 0; dd < D; ++dd) sums[j][dd] += partial_sums[t][j * D + dd];
        }
    }
    Mat newc = centroids;
    for (int j = 0; j < k; ++j) {
        if (counts[j] > 0) {
            for (int dd = 0; dd < D; ++dd) newc[j][dd] = sums[j][dd] / counts[j];
        }
    }
    double shift = 0.0;
    for (int j = 0; j < k; ++j) shift = max(shift, sqrt(sq_dist(newc[j], centroids[j])));
    centroids.swap(newc);
    if (shift <= tol) break;
}
double inertia = 0.0;
for (int i = 0; i < N; ++i) inertia += sq_dist(data[i], centroids[labels[i]]);
auto t1 = clock::now();
double elapsed = chrono::duration<double>(t1 - t0).count();
return {centroids, labels, inertia, elapsed};
}

double centroid_l2_diff(const Mat &a, const Mat &b) {
    double s = 0.0;
    for (size_t i = 0; i < a.size(); ++i) s += sqrt(sq_dist(a[i], b[i]));
    return s;
}

int main(int argc, char **argv) {
    string data_path = "data_students.csv";
    string init_path = "init_centroids.csv";
    int max_iter = 100;
    double tol = 1e-4;
    if (argc > 1) {
        for (int i = 1; i < argc; ++i) {
            string s = argv[i];
            if (s == "--data" && i + 1 < argc) data_path = argv[++i];
            else if (s == "--init" && i + 1 < argc) init_path = argv[++i];
            else if (s == "--max-iter" && i + 1 < argc) max_iter = stoi(argv[++i]);
            else if (s == "--tol" && i + 1 < argc) tol = stod(argv[++i]);
        }
    }
    auto data = read_csv(data_path);
    auto init = read_csv(init_path);
    if (data.empty() || init.empty()) { cerr << "Empty data or init file\n"; return 1; }
    int k = (int)init.size();
    cout << "Data points: " << data.size() << ", dims=" << data[0].size() << ", k=" << k <<

```

```

"\n";
// sequential
cout << "Running sequential K-Means...\n";
auto res_seq = kmeans_sequential(data, init, max_iter, tol);
cout << " Seq time: " << res_seq.elapsed << " s, inertia=" << res_seq.inertia << "\n";
// parallel OpenMP
cout << "Running OpenMP-parallel K-Means...\n";
auto res_par = kmeans_openmp(data, init, max_iter, tol);
cout << " Par time: " << res_par.elapsed << " s, inertia=" << res_par.inertia << "\n";
double speedup = res_seq.elapsed / res_par.elapsed;
cout << "Speedup (seq/par): " << speedup << "\n";
double cdiff = centroid_l2_diff(res_seq.centroids, res_par.centroids);
cout << "Centroid L2 sum difference: " << cdiff << "\n";
// Interpret clusters by centroid means
vector<pair<double, int>> means;
for (int j = 0; j < k; ++j) {
    double m = accumulate(res_seq.centroids[j].begin(), res_seq.centroids[j].end(), 0.0) /
res_seq.centroids[j].size();
    means.push_back({m, j});
}
sort(means.begin(), means.end());
vector<string> labels = {"Low", "Average", "High"};
cout << "Cluster interpretations:\n";
for (size_t i = 0; i < means.size(); ++i) {
    string lab = (i < labels.size() ? labels[i] : "Cluster");
    cout << " cluster " << means[i].second << " -> " << lab << ", mean=" << means[i].first
<< "\n";
}
return 0;
}

```

How to compile: g++ -fopenmp -O2 Practical5.cpp -o Practical5

How to run: Practical5 --data data_students.csv --init init_centroids.csv

OUTPUT:

```

C:\Windows\System32\cmd.e  X  +  v
Microsoft Windows [Version 10.0.26200.8246]
(c) Microsoft Corporation. All rights reserved.

C:\D_Drive\B.E\Sem-VII\LP-V\HPC>g++ -fopenmp -O2 Practical5.cpp -o Practical5

C:\D_Drive\B.E\Sem-VII\LP-V\HPC>Practical5 --data data_students.csv --init init_centroids.csv
Data points: 10000, dims=3, k=3
Running sequential K-Means...
Seq time: 0.0030695 s, inertia=1.87975e+06
Running OpenMP-parallel K-Means...
Par time: 0.0085799 s, inertia=1.87975e+06
Speedup (seq/par): 0.357755
Centroid L2 sum difference: 3.76174e-13
Cluster interpretations:
cluster 0 -> Low, mean=39.91
cluster 2 -> Average, mean=64.9692
cluster 1 -> High, mean=84.9286
C:\D_Drive\B.E\Sem-VII\LP-V\HPC>

```